

SIMATS ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

ASSESSMENT TOOL 2 — CASE STUDY REPORT

Course Name & Code:

Cryptography and Network Security (CSA51)

Course Outcome Covered:

CO3: Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks (BL4, BL5)

Student Name:

T.Lakshmi Prasad Naidu

Register Number:

192324187

Code of Conduct Certification:

I, T.Lakshmi Prasad Naidu (Reg No: 192324187), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Case Study 1: Weak Password Hashing in Web Application

Case Scenario: A web application stores user passwords using unsalted SHA-1 hashes. After a breach, attackers quickly crack multiple passwords using precomputed tables.

Vulnerability Analysis

- **Impact of Lack of Salting:** Without a salt, identical plain-text passwords produce identical SHA-1 output hashes across all users. Attackers utilize precomputed lookup tables (Rainbow Tables) to reverse millions of hashes into plaintext in seconds without executing real-time cryptographic computations.
- **Inherent Cryptographic Flaws in SHA-1:** SHA-1 is a general-purpose, high-speed digest algorithm engineered for rapid throughput, not password storage. Modern consumer GPUs and dedicated ASICs can compute tens of billions of SHA-1 hashes per second, making offline dictionary and brute-force attacks trivial once the hash database is leaked.

Recommended Secure Password Hashing Strategy

- **Recommended Primitive:** Migrate immediately to Argon2id (winner of the Password Hashing Competition) or bcrypt / PBKDF2-HMAC-SHA256 with a unique, cryptographically secure 128-bit salt per user (generated via `os.urandom` or `CSPRNG`).
- **Justification:** Argon2id incorporates both memory-hardness (configuring RAM requirements) and CPU-time hardness (iterations), effectively disabling GPU/ASIC parallel cracking hardware. Per-user

salting renders precomputed rainbow tables useless by forcing attackers to generate new lookup tables for every target account.

Case Study 2: Digital Signature Compromise in Financial System

Case Scenario: *A financial institution uses digital signatures for transaction approval. An attacker gains access to a user's private key and performs unauthorized transactions.*

Impact Evaluation

- **Breakdown of Authentication:** Digital signatures rely on asymmetric cryptography where the private key uniquely identifies the signer. Private key compromise completely destroys identity assurance—the system can no longer distinguish legitimate approvals from fraudulent attacker actions.
- **Destruction of Non-Repudiation:** Non-repudiation assumes that only the private key owner could have produced the signature. Once key exposure is established, legal non-repudiation fails: legitimate users can claim valid transactions were fraudulent, while attackers can claim unauthorized transactions were authentic user actions.

Mitigation Strategies & Safeguards

- **Hardware Security Modules (HSMs) & TPMs:** Store user and system private keys inside dedicated Hardware Security Modules (HSM) or Trusted Platform Modules (TPM) where keys are marked non-exportable and cryptographic operations occur within tamper-resistant hardware boundaries.
- **Multi-Factor Authorization & Multi-Sig Protocols:** Enforce dual-custody / multi-signature requirements for high-value financial transfers, requiring sign-off from multiple keys and step-up Out-of-Band (OOB) multi-factor authentication (e.g., FIDO2/WebAuthn hardware keys).
- **Automated Key Revocation & Short Validity Windows:** Deploy Online Certificate Status Protocol (OCSP) Stapling to immediately invalidate compromised certificates upon detection.

Case Study 3: Kerberos Authentication Failure in Distributed System

Case Scenario: *A distributed system using Kerberos experiences frequent authentication failures due to time synchronization issues between servers.*

Technical Analysis of Time Synchronization in Kerberos

- **Role of Timestamps in Ticket Validation:** Kerberos relies on synchronized system clocks to prevent Replay Attacks without requiring the Key Distribution Center (KDC) to store stateful databases of all issued tickets. Every Ticket Granting Ticket (TGT) and Service Ticket contains an issuance timestamp and lifetime.

- **Mechanism of Failure:** If the system clock skew between the Client, KDC, and Target Application Server exceeds the allowed threshold (typically 5 minutes / 300 seconds), ticket requests are rejected with a KRB_AP_ERR_SKEW error, causing widespread authentication lockout.

Proposed Solutions & Network Architecture

- **Resilient Network Time Protocol (NTP) Hierarchy:** Implement an enterprise NTP infrastructure referencing multiple Stratum-1 time servers (atomic/GPS-synced). Domain controllers and application servers must synchronize periodically to guarantee time skew remains strictly below 1 second.
- **Automated Clock Drift Monitoring & Alerting:** Deploy active monitoring agents across all nodes to trigger alerts whenever time skew relative to NTP stratum servers exceeds 30 seconds, preventing authentication disruptions before reaching the Kerberos failure threshold.

Case Study 4: Certificate Authority Breach in PKI System

Case Scenario: A Certificate Authority (CA) is compromised, leading to the issuance of fake X.509 certificates for trusted domains.

Impact Evaluation on PKI & Secure Communication

- **Collapse of Transitive Trust:** Public Key Infrastructure relies on hierarchical trust anchored by CAs. A single compromised CA enables attackers to issue cryptographically valid certificates for any high-value domain (e.g., google.com, bank.com).
- **Man-in-the-Middle (MitM) Risk:** Attackers can position themselves on the network path, present legitimate-appearing fake certificates, terminate TLS/SSL encryption seamlessly, and intercept/decrypt user credentials and financial payloads without triggering browser security warnings.

Detection & Mitigation Mechanisms

- **Certificate Transparency (CT) Logs:** Mandate append-only, publicly verifiable cryptographic ledgers for all issued certificates. Domain owners monitor CT logs continuously to detect fraudulently issued certificates within minutes.
- **DNS Certification Authority Authorization (CAA) Records:** Configure DNS CAA records specifying precisely which CAs are permitted to issue certificates for the domain, causing compliant CAs to refuse illegitimate issuance requests.
- **Immediate Revocation & Root CA Distrust:** Publish revoked CAs across Certificate Revocation Lists (CRLs) and push immediate browser vendor updates to distrust compromised intermediate or root CA certificates.

Case Study 5: Integrity Verification in Cloud Data Storage

Case Scenario: A cloud service provider uses hashing to verify file integrity, but users report data

tampering incidents. Investigation reveals improper hash validation practices.

Analysis of Integrity Implementation Failures

- **Flaw 1 — Plain Hashing vs. Keyed Authentication:** A standard cryptographic hash $H(M)$ provides integrity against unmalicious bit corruption, but does NOT provide authenticity. If an attacker modifies file M to M' and overwrites stored hash $H(M)$ with $H(M')$, naive validation passes successfully.
- **Flaw 2 — Improper Client/Server Validation Boundaries:** Performing integrity checks solely client-side or trusting unauthenticated metadata allows attackers to intercept transmission channels and tamper with both data payloads and checksums concurrently.

Robust Integrity Verification Mechanism & Justification

- **Recommended Solution:** Implement HMAC-SHA256 (Hash-based Message Authentication Code) or Asymmetric Digital Signatures (ECDSA) alongside Authenticated Encryption modes like AES-256-GCM.
- **Justification:** HMAC combines the file hash with a secret key known only to authorized cloud tenants. An attacker modifying cloud files cannot re-generate a matching HMAC tag without possessing the private/secret key, guaranteeing both Data Integrity and Data Authenticity.